

■ Lab 5 — Outputs & Variables (GitPod)

■ Objectifs du Lab

- Organiser le code selon les **bonnes pratiques** Terraform
- Exporter des attributs avec les **outputs**
- Déclarer et utiliser des **variables** avec validation
- Assigner via **CLI**, **tfvars** et **TF_VAR_**

■ Étape 1 — Préparer l'environnement

Reprenez votre environnement GitPod → <https://gitpod.io/workspaces>

```
cd labs/lab5-outputs
```

■ Étape 2 — Créer les fichiers (bonnes pratiques)

Créez chaque fichier dans l'éditeur VS Code GitPod :

`backend.tf`

```
terraform {
  backend "s3" {
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
```

`versions.tf`

```
terraform {
  required_version = ">= 1.15.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
```

`provider.tf`

```
provider "aws" {  
  region = var.region  
}
```

`variables.tf`

```
variable "username" {  
  description = "Votre prenom"  
  type        = string  
  validation {  
    condition     = length(var.username) >= 2 && length(var.username) <= 20  
    error_message = "Le username doit contenir entre 2 et 20 caractères."  
  }  
}  
  
variable "region" {  
  description = "Region AWS"  
  type        = string  
  default     = "eu-west-1"  
}  
  
variable "instance_type" {  
  description = "Type d'instance EC2"  
  type        = string  
  default     = "t2.micro"  
  validation {  
    condition     = contains(["t2.micro", "t2.small", "t2.medium"], var.instance_type)  
    error_message = "Le type d'instance doit être t2.micro, t2.small ou t2.medium."  
  }  
}  
  
variable "environment" {  
  description = "Environnement"  
  type        = string  
  default     = "dev"  
  validation {  
    condition     = contains(["dev", "prod"], var.environment)  
    error_message = "L'environnement doit être dev ou prod."  
  }  
}  
  
}
```

`main.tf`

```
resource "aws_security_group" "lab5_sg" {
  name          = "lab5-sg-${var.username}-${var.environment}"
  description    = "Security group Lab 5 - ${var.username}"
  tags = { Name = "lab5-sg-${var.username}", Lab = "lab5", Username = var.username, Environment = var.environment }
}

resource "aws_instance" "lab5_instance" {
  ami              = "ami-0694d931cee176e7d"
  instance_type    = var.instance_type
  vpc_security_group_ids = [aws_security_group.lab5_sg.id]
  tags = { Name = "lab5-instance-${var.username}", Lab = "lab5", Username = var.username, Environment = var.environment }
}
```

`outputs.tf`

```
output "instance_id" { description = "ID de l'instance EC2" ; value = aws_instance.lab5_instance.id }
output "instance_public_ip" { description = "IP publique" ; value = aws_instance.lab5_instance.public_ip }
output "security_group_id" { description = "ID du SG" ; value = aws_security_group.lab5_sg.id }
output "security_group_name" { description = "Nom du SG" ; value = aws_security_group.lab5_sg.name }
output "deployment_summary" {
  description = "Résumé du déploiement"
  value = { username = var.username, environment = var.environment, instance = aws_instance.lab5_instance.id }
}
```

■ Étape 3 — Init avec Backend S3

```
BUCKET_NAME=$(grep "bucket" $HOME/tf-training-info.txt | awk '{print $NF}')
```

```
terraform init \
  -backend-config="bucket=${BUCKET_NAME}" \
  -backend-config="key=terraform.tfstate"
```

■ Étape 4 — Les 3 façons d'assigner les variables

Méthode 1 — CLI flag

```
terraform apply -var="username=<votre-prenom>" -var="environment=dev"
```

Méthode 2 — tfvars

Créez `terraform.tfvars` dans l'éditeur :

```
username      = "<votre-prenom>"
environment   = "dev"
instance_type = "t2.micro"
```

```
terraform apply
```

Méthode 3 — TF_VAR_

```
export TF_VAR_username="<votre-prenom>"
export TF_VAR_environment="dev"
terraform apply
```

■ Étape 5 — Tester la validation

```
terraform plan -var="username=<votre-prenom>" -var="environment=staging"
terraform plan -var="username=<votre-prenom>" -var="instance_type=t3.large"
```

■ Étape 6 — Utiliser terraform output

```
terraform output
terraform output instance_id
terraform output -json deployment_summary
terraform output -raw instance_id
```

■ Étape 7 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

■ Validation du Lab

#	Critère	Validé
1	Fichiers séparés : `backend.tf`, `versions.tf`, `provider.tf`	■
2	`terraform init -backend-config` réussi	■
3	Validation bloque `environment=staging`	■
4	Les 3 méthodes d'assignation fonctionnent	■
5	`terraform output -json` affiche un objet JSON	■
6	`terraform destroy` supprime les ressources	■

■ ****Fin du Lab 5 !****